Here is your battle plan for Stanford TreeHacks. This is designed for speed, stability, and maximum judge impact.

1. Historical Wildfire Data Sources

For a hackathon, **do not** try to scrape PDF reports. Use these pre-structured datasets.

Dataset Name	Best For...	Content	Access	Format
ICS-209-PLUS	**The Holy Grail.** This is your primary database.	35k+ incidents (1999-2020) with daily "snapshots" of fire behavior, resources used, structures threatened/destroyed, and cost.	GitHub Link	CSV / JSON
CAL FIRE Redbooks	Outcome data	Annual summaries of large fires (>300 acres) with specific cause and damage stats.	CAL FIRE Data	CSV / PDF
MTBS	Fire Behavior	Burn severity boundaries. Good for visualizing "what happened" in the Analyze tab.	MTBS.gov	GeoJSON / Shapefile
Synoptic Data	Weather Context	Historical wind/temp/humidity data if	Synoptic API	API (Free Tier)

		the fire record is missing weather info.		

Hackathon Strategy:

- **Download the ICS-209-PLUS CSV.**
- Filter it immediately: State = CA, Final Size > 100 acres.
- Convert this filtered subset (~2,000 rows) into your JSON "knowledge base."
- *Ignore* geospatial shapefiles for the backend logic; just use the text/numeric fields for matching.

---

2. Tech Stack: The "Vibe-Coder" Speed Run

This stack minimizes configuration. You want "Serverless" everything so you don't manage infrastructure.

- **Coding Assistant: Cursor** (Use "Composer" mode to write multi-file code at once).
- **Frontend: Next.js** (React) deployed on **Vercel**.
  - *Why:* It's the standard; Vercel handles deployment instantly.
- **Backend / Database: Supabase**.
  - *Why:* It gives you a Postgres database, Auth, and **Vector Search** in one dashboard. No separate Pinecone account needed.
- **Voice Input: Deepgram Nova-2**.
  - *Why:* It is significantly faster (<300ms) and handles outdoor wind noise better than OpenAI Whisper.
- **LLM (Logic & RAG): Anthropic Claude 3.5 Sonnet** (via Vercel AI SDK).
  - *Why:* Best reasoning for "tactical critique" and extraction.
- **Maps: Mapbox GL JS**.
  - *Why:* Looks 10x better than Google Maps for dark-mode tactical views.

---

3. Voice-to-Structured-Data Pipeline

The Architecture:

User Voice -> Deepgram (WebSocket) -> Text -> Claude 3.5 (Entity Extraction) -> JSON Object

Step-by-Step Implementation:

1. **Frontend:** Use the browser's MediaRecorder API to capture audio chunks.
2. **Transcribe:** Stream these chunks to **Deepgram** via a WebSocket.
   - *Hack:* Don't do "real-time streaming" if it's too hard. A "Push to Talk" button that sends a single blob to Deepgram's API is easier to debug and fine for a prototype.
3. **Extract:** Send the transcript to Claude with a system prompt like:

    - *"You are a wildland fire tactical aide. Extract these entities: Fuel Model (e.g., Grass, Brush), ROS (Rate of Spread), Threat (Structures, Powerlines). Return ONLY JSON."*
4. **Latency Goal:** < 2 seconds total.

---

4. RAG Implementation (The "Similar Fire" Engine)

Simplest Path (Supabase + Vercel AI SDK):

1. **Prepare Data:** Take your filtered ICS-209 JSON. Create a "narrative" string for each fire combining its fields:
    - "Fire: Tubbs. Fuel: Oak/Grass. Wind: 40mph NE. Terrain: Steep Canyons. Behavior: Extreme spotting."
2. **Embed:** Run a script to generate embeddings for these strings using **OpenAI's `text-embedding-3-small`** (cheap & fast).
3. **Store:** Upload these vectors to Supabase.
4. **Query:** When the IC speaks ("Wind driven fire in oak woodland..."), embed their text and query Supabase for the top 5 closest matches.

Hackathon Shortcut (Context Stuffing):

If your filtered dataset is < 500 fires, you might not *need* a vector DB. You can dump the summary of all 500 fires into Claude's context window (200k tokens) and just ask: *"Here is a list of 500 historical fires. Which ones match this current situation?"*

- **Verdict:** Try Context Stuffing first. If it's too slow/expensive, switch to Supabase Vectors.

---

5. Historical Alignment Score Methodology

This is your "Secret Sauce" for the judges. Don't just make up a number; use a weighted heuristic.

The Formula:

Redhue calculates a **0-100 Score** based on 3 vectors:

1. **Situational Similarity (40%):** How close is the current fire's fuel/weather to the historical match? (Cosine Similarity score).
2. **Tactical Match (40%):** Does the user's proposed plan (e.g., "Direct attack with dozers") match successful tactics used in those similar fires?
    - *Logic:* If "Dozer Line" worked in the matched fire, and User says "Dozer Line", +Points.
3. **Negative Outcome Penalty (20%):** Did the similar fires result in injuries or escapes? If yes, lower the score to warn the user.

Display:

Show a gauge (Green/Yellow/Red).

- **Text:** "78% Alignment with Historical Success."
- **Reasoning:** "High alignment on tactics, but historical data suggests adding air tanker support due to steep terrain failures in the 2017 Thomas Fire."

---

6. UI/UX for High Stress

Design Rules:

- **The "3-Second Rule":** Any info not readable in 3 seconds is useless.
- **Dark Mode Default:** Firefighters work at night; bright white screens ruin night vision.
- **Button Size:** Minimum 60px height (easy to tap with gloves).
- **Typography:** Use **Inter** or **Roboto Mono**. High contrast (White text on pure black).

Key Screens:

1. **"Listen" Mode:** A massive, pulsing microphone button. No other clutter.
2. **"Situation" Card:** Large tags for extracted entities (FUEL: BRUSH, WIND: 25MPH).
3. **"Analogous Fires":** A carousel of cards: "Similar Event: 2003 Cedar Fire".
4. **"Analyze" Output:** The Alignment Score gauge + 3 bullet points of "Critical Considerations."

---

7. Competitor Deep Dive

Competitor	Core Function	Why Redhue Wins (The Gap)
Tablet Command	Drag-and-drop unit tracking on a map.	It's manual data entry. It doesn't *think* or give advice.
WIFIRE / FIRIS	Predictive modeling (where the fire *will go*).	It predicts spread, but doesn't tell the IC *what to do* about it.
ATAK	Blue force tracking (military style).	Extremely complex/clunky UI. High learning curve.
Redhue	**Decision Support.**	The only tool using *voice* to bridge *prediction* and *action*.

---

8. 36-Hour Build Plan

- **Friday Night (Hours 0-6): Data & Setup**
  - Download ICS-209 data. Clean it.
  - Init Next.js repo. Connect Supabase.
  - *Goal:* A blank web page that connects to a DB.
- **Saturday Morning (Hours 6-14): The Voice Pipeline**
  - Build the "Push to Talk" button.
  - Connect Deepgram.
  - Connect Claude for Entity Extraction.
  - *Goal:* You speak, and JSON appears on the screen.
- **Saturday Afternoon (Hours 14-22): The Brain (RAG)**
  - Implement the vector search (or context stuffing).
  - Feed the "Situation JSON" into the search.
  - *Goal:* You speak, and the app shows "Matches: Tubbs Fire, Atlas Fire."
- **Saturday Night (Hours 22-28): The Analyze Tab**
  - Build the Alignment Score logic.
  - Refine the prompt to generate "Reasoning."
  - *Goal:* The "Critique" feature works.
- **Sunday Morning (Hours 28-36): Polish & Pitch**
  - CSS cleanup (Dark mode!).
  - **Fake the data if needed:** If the DB breaks, hardcode 3 perfect demo scenarios.
  - Practice the pitch.

---

9. Demo & Pitch Script Outline (3 Minutes)

1. **The Hook (30s):** "In the first 3 hours of a wildfire, the Incident Commander is blind. They are processing 1,000 inputs under extreme stress. Mistakes here cost lives."
2. **The Solution (30s):** "Meet Redhue. It's an AI Chief of Staff. It listens to the radio traffic, understands the fire, and instantly finds matching historical events."
3. **The Demo (1m 30s):**
   - *Action:* Hold phone, press button. Speak: *"We have a wind-driven fire in heavy brush, spotting 200 yards ahead, threatening the ridge."*
   - *Reveal:* App instantly extracts tags. Shows "Match: 2007 Witch Fire."
   - *Action:* Click Analyze. "I plan to go direct with engines."
   - *Reveal:* App shows **Yellow Score**. "Caution: Direct attack failed in similar 2007 conditions. Recommend indirect dozer lines."
4. **The Ask/Vision (30s):** "We aren't replacing the commander; we're giving them the wisdom of 10,000 past fires. We are Redhue."

10. Budget (Hackathon Scale)

- **Vercel:** Free (Hobby Tier).
- **Supabase:** Free (Hobby Tier).
- **Deepgram:** Free ($200 credit for new users usually available).
- **Claude API:** Use Hackathon credits.
- **Mapbox:** Free tier is generous.

Go build this. It addresses a real gap with current tech. Good luck at TreeHacks!